



### Actividades prácticas: Asociación y Dependencia

#### Objetivos

- Implementar o completar la implementación de dos o más clases relacionadas, a partir de una especificación que incluye un diagrama de clases, considerando las responsabilidades asignadas a cada clase.
- Verificar los servicios a través de una clase tester considerando las responsabilidades asignadas a cada clase.
- Dibujar un diagrama de objetos de clases asociadas.
- Modificar la implementación de una clase a partir de un cambio en el diseño y observar en qué clases impacta el cambio.

**EJERCICIO 1.** En un videojuego cada criatura dispone de cierto nivel de energía y tiene un refugio. En un momento dado, una criatura puede estar dormida o despierta y ha realizado, desde la última vez que durmió, cierta cantidad de caminatas. Implemente la clase *Criatura* y verifique de acuerdo a la especificación:

| Refugio                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Criatura                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>&lt;&lt;Atributos de clase&gt;&gt; capacidadAlacena = 20 cantidadCamas = 10 &lt;&lt;Atributos de instancia&gt;&gt; alimentos, bebidas, camas: entero</pre>                                                                                                                                                                                                                                                                                                                                                                                                            | <pre>&lt;&lt;Atributos de clase&gt;&gt; maxEnergia = 100 minEnergia = 0 consumoEnergia = 10 &lt;&lt;Atributos de instancia&gt;&gt; energia: entero refugio: Refugio despierto: boolean caminatas: entero</pre>                                                                                                                                                                               |
| <pre>&lt;&lt;Constructor&gt;&gt; Refugio(a, b, c: entero) &lt;&lt;Comandos&gt;&gt; consumirAlimento() consumirBebida() ocuparCama(): boolean desocuparCama(): boolean reponerAlimentos(n: entero): boolean reponerBebidas(n: entero): boolean &lt;&lt;Consultas&gt;&gt; obtenerAlimentos(): entero obtenerBebidas(): entero obtenerCamas(): entero obtenerCapacidadAlacena(): entero esHabitable(): boolean disponibilidad(): entero diasSupervivencia(): entero mayorAlimentos(r: Refugio): boolean equals(c: Refugio): boolean clone(): Refugio toString(): String</pre> | <pre>&lt;&lt;Constructor&gt;&gt; Criatura(r: Refugio) &lt;&lt;Comandos&gt;&gt; comer(): boolean beber(): boolean dormir(): boolean despertar(): boolean caminar(): boolean &lt;&lt;Consultas&gt;&gt; obtenerEnergia(): entero obtenerCaminatas(): entero obtenerHumor(): entero obtenerRefugio(): Refugio estaDormido(): boolean mayorEnergia(c: Criatura): boolean toString(): String</pre> |

- El constructor crea una criatura con el máximo de energía, asociada al refugio *r*, que está despierta y aun no caminó. Requiere *r* ligada.
- Cuando la criatura recibe el mensaje comer o beber, controla que la criatura no esté dormida y que haya alimentos o bebidas en su refugio, según corresponda. Si así es, su nivel de energía aumenta en 1 (sin exceder nunca el máximo permitido) y además se consume un alimento o bebida del refugio.
- Cuando una criatura recibe el mensaje dormir, si está despierta y hay camas disponibles en su refugio, la ocupa y cambia su estado.
- Cuando una criatura recibe el mensaje despertar estando dormida, cambia su estado, desocupa una cama en el refugio y reestablece en 0 el atributo de caminatas realizadas sin dormir.
- Cuando la criatura recibe el mensaje caminar, si está despierta y tiene suficiente energía para hacerlo, ésta se decrementa de acuerdo a consumoEnergia. El atributo caminatas registra cuántas veces caminó sin haber dormido, pudiendo una criatura caminar hasta 3 veces sin dormir. Si recibe el mensaje caminar por cuarta vez, si hay camas disponibles ocupa una y se pone a dormir; si no hay camas disponibles, su energía toma el valor mínimo, permaneciendo caminatas en 3.
- Todos los comandos retornan true si la acción pudo ejecutarse, y falso en caso contrario.
- Una criatura manifiesta diferentes grados de humor, los que expresa mediante un valor numérico que va de 1 a 3, siendo 1 el más infeliz y 3 el más contento. El humor depende del nivel de energía y del estado del refugio. Si el refugio no es habitable el humor es 1, no importa cuál sea la energía; si es habitable, el humor queda determinado por la siguiente tabla.

| Energía | Humor |
|---------|-------|
| 0...40  | 1     |
| 41...70 | 2     |
| >70     | 3     |

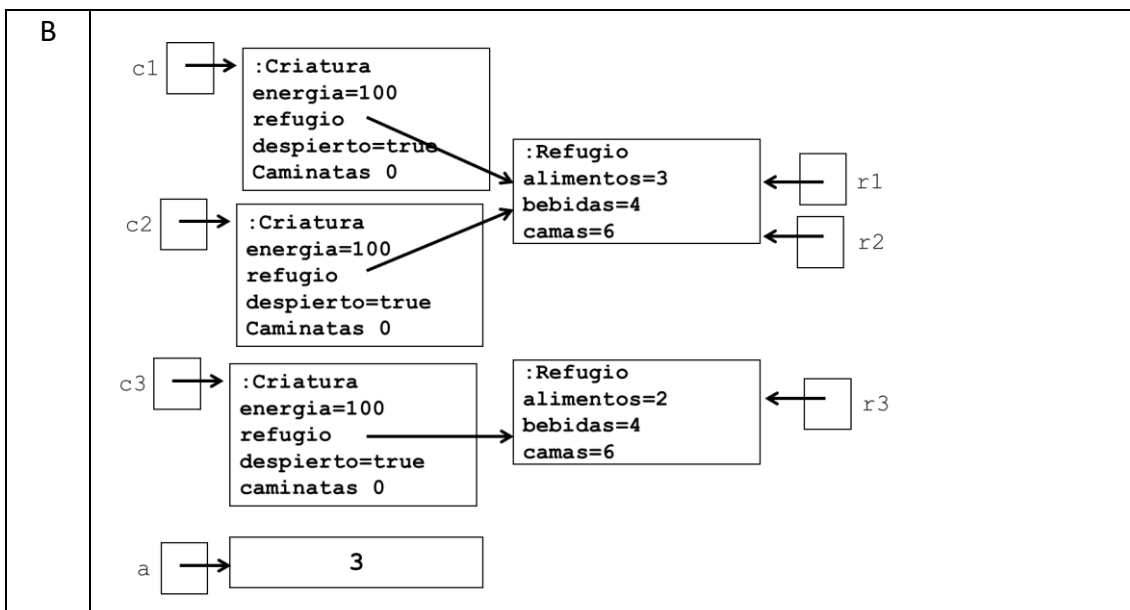
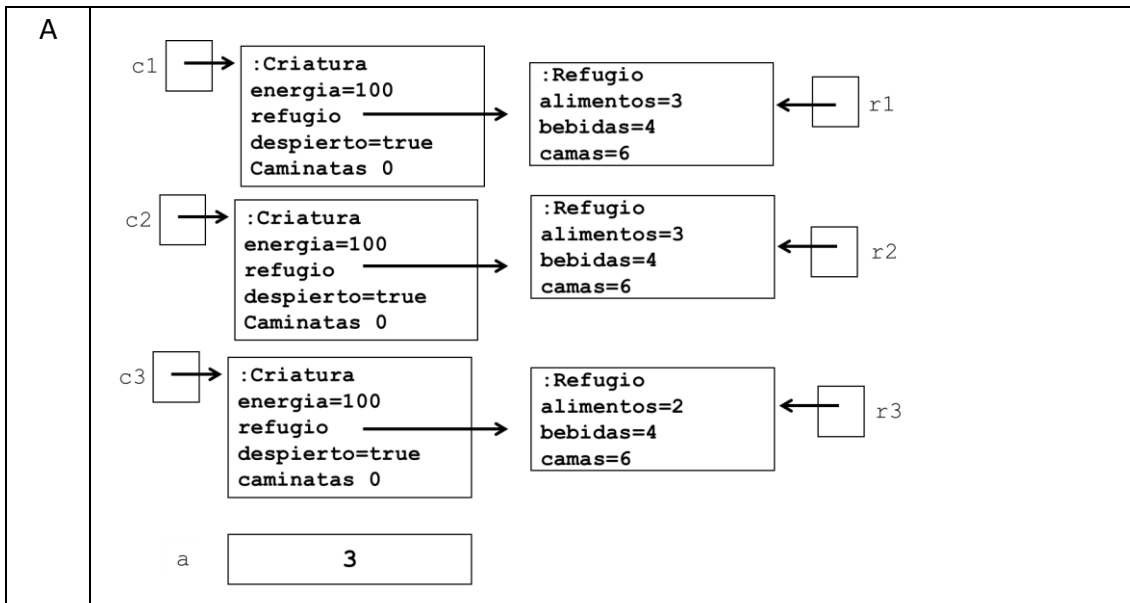
**EJERCICIO 2.** Dadas las clases *Refugio* y *Criatura* implementadas antes, indique qué diagrama corresponde a la siguiente secuencia de instrucciones:

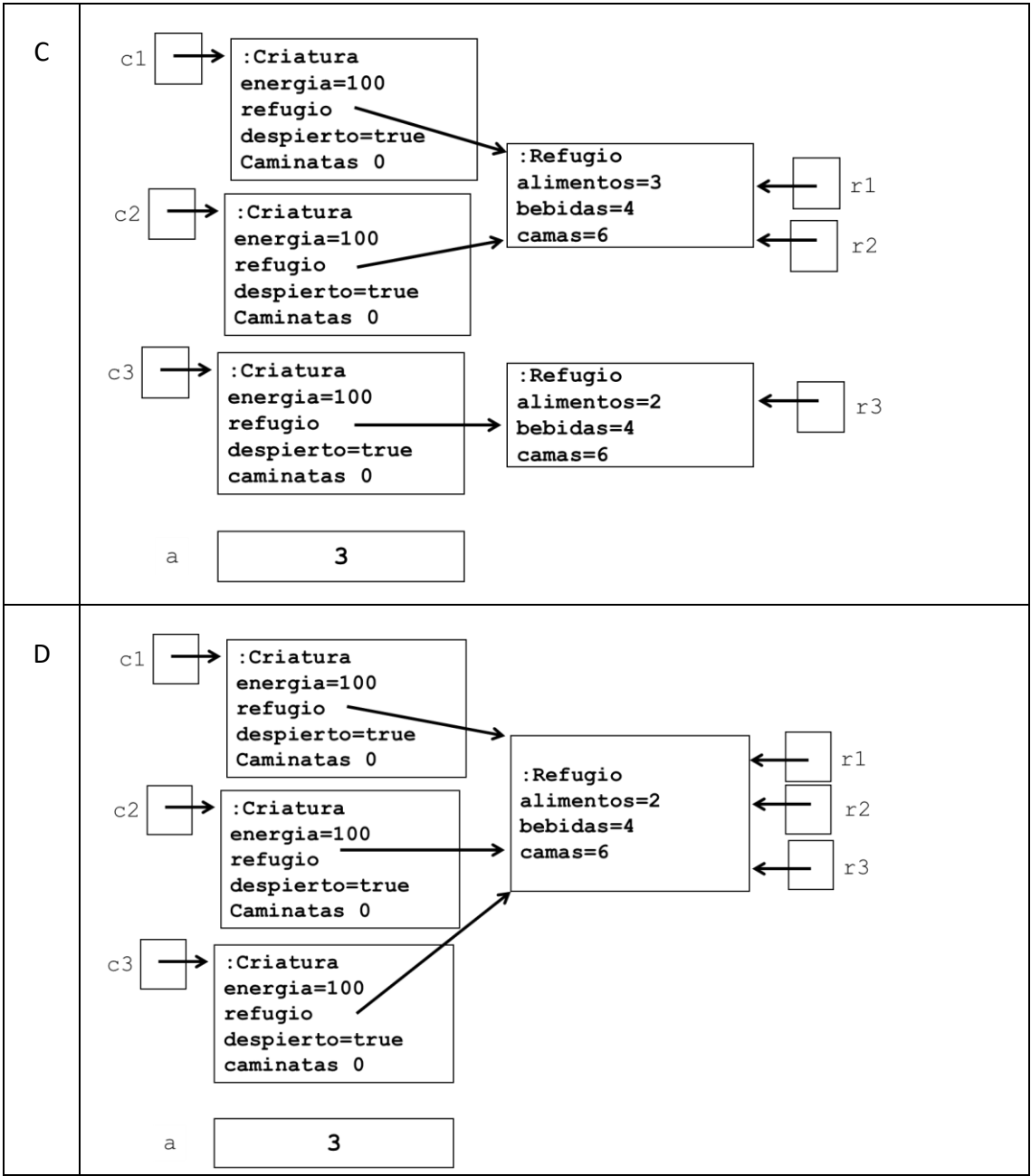
```
Refugio r1 = new Refugio(3,4,6);
Refugio r2 = r1;
Refugio r3 = r1.clone();
Criatura c1 = new Criatura (r1);
Criatura c2 = new Criatura (r2);
Criatura c3 = new Criatura (r3);
r3.consumirAlimento();
int a = r2.obtenerAlimentos();
```



# Introducción a la Programación Orientada a Objetos

## DCIC – UNS – 2025







Introducción a la Programación Orientada a Objetos  
DCIC – UNS – 2025

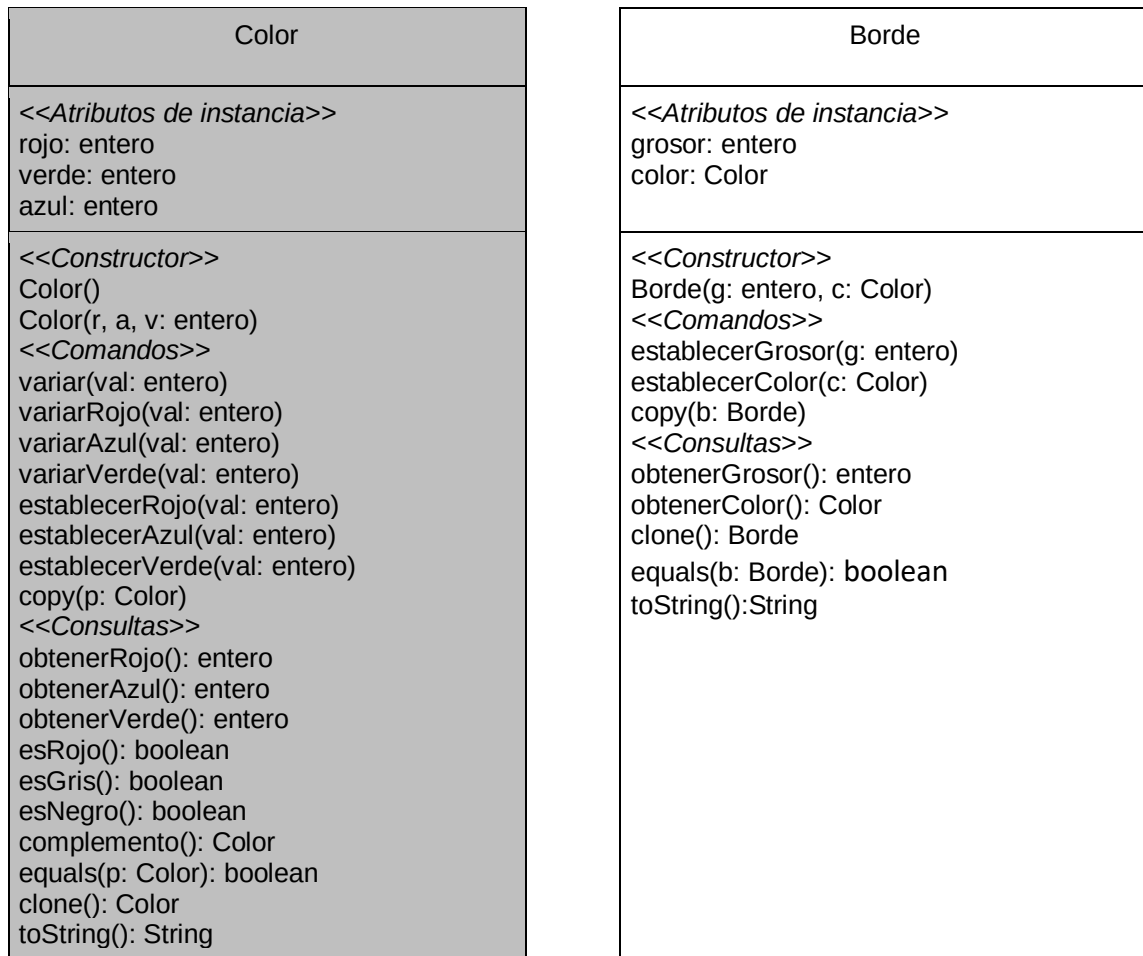


**EJERCICIO 3.** Dado el siguiente diagrama de clases y la clase *Libro* publicada, implemente la clase *Prestamo* y una clase *Tester* que la verifique.

| Fecha                                                                                                                                                                                                                                                                                                                                                                               | Prestamo                                                                                                                                                                                                                                                                                                                                                                | Libro                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <<Atributos de instancia>><br>dia: entero<br>mes: entero<br>anio: entero                                                                                                                                                                                                                                                                                                            | <<Atributos de instancia>><br>libro: Libro<br>socio: String<br>fechaPrestamo: Fecha<br>fechaDevolucion: Fecha<br>devuelto:boolean                                                                                                                                                                                                                                       | <<Atributos de instancia>><br>nombre: String<br>autor: String<br>editorial: String<br>categoria: char                                                                                                                                                                                                                                                    |
| <<Constructor>><br>Fecha(d, m, a: entero)<br><<Comandos>><br>establecerDia(d: entero)<br>establecerMes(m: entero)<br>establecerAnio(a: entero)<br><<Consultas>><br>obtenerDia(): entero<br>obtenerMes(): entero<br>obtenerAnio(): entero<br>esBisiesto():boolean<br>equals(f: Fecha): boolean<br>esAnterior(f: Fecha): boolean<br>mismoAnio(f: Fecha):boolean<br>toString(): String | <<Constructor>><br>Prestamo(l: Libro, fp: Fecha, fd: Fecha, s:String)<br><<Consultas>><br>obtenerLibro(): Libro<br>obtenerFechaPrestamo(): Fecha<br>obtenerFechaDevolucion(): Fecha<br>estaDevuelto():boolean<br>obtenerSocio(): String<br>estaAtrasado(hoy: Fecha): boolean<br>masAntiguo(p: Prestamo): Prestamo<br>equals(p: Prestamo): boolean<br>toString(): String | <<Constructor>><br>Libro(n, a, e: String, c: char)<br><<Comandos>><br>establecerNombre(p: String)<br>establecerAutor(p: String)<br>establecerEditorial(p: String))<br>establecerCategoria(p: char)<br><<Consultas>><br>obtenerNombre(): String<br>obtenerAutor(): String<br>obtenerEditorial(): String<br>obtenerCategoria(): char<br>toString(): String |

- *Prestamo(l: Libro, fp: Fecha, fd: Fecha, s:String)*. Requiere l, fp, fd y s ligados. Inicialmente el libro se encuentra prestado. fechaDevolucion refiere a la fecha en la que se debe devolver el libro.
- *estaAtrasado(hoy: Fecha): boolean*. Recibe como parámetro la fecha actual y retorna verdadero si el libro no se devolvió y la fecha de hoy es posterior a la fecha de devolución.
- *masAntiguo(p: Prestamo): Prestamo*. Retorna el préstamo con la fecha del préstamo más antigua.
- *equals(p: Prestamo): boolean*. Aplica igualdad superficial por libro, y en profundidad por socio, fechaPrestamo y fechaDevolucion.

**EJERCICIO 4.** Dado el siguiente diagrama que modela la clase *Borde* asociada a la clase *Color*:



- Implemente la clase *Borde*. Los métodos *copy*, *clone* e *equals* se implementan en forma superficial. Los métodos *copy* y *equals* asumen el parámetro ligado.
- Dibuje el diagrama de objetos al terminar el bloque de asignaciones del siguiente segmento:

```

Color c1, c2, c3, c4;
Borde b1, b2, b3, b4, b5, b6, b7;
c1 = new Color(110, 110, 110);
c2 = new Color(110, 110, 110);
c3 = c1;
c4 = c1.clone();
b1 = new Borde(1, c1);
b2 = new Borde(1, c2);
b3 = b2.clone();
b4 = new Borde(b2.obtenerGrosor(), b2.obtenerColor());
b5 = b2;
b6 = new Borde(1, c3);
b7 = new Borde(1, c4);
  
```



## Introducción a la Programación Orientada a Objetos DCIC – UNS – 2025

- c. Muestre la salida de las siguientes instrucciones luego de haberse ejecutado el segmento de código del inciso anterior.

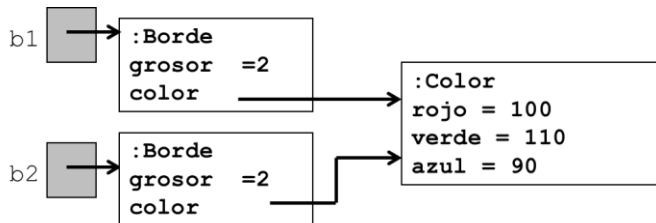
```
System.out.println(c1 == c2+" "+ c1 == c3+" "+ c1 == c4);
System.out.println( c1.equals(c2) +" "+
c1.equals(c3) +" "+ c1.equals(c4));
System.out.println( b2 == b3+" "+ b2 == b4+" "+
b2 == b5+" "+ b2 == b6+" "+ b2 == b7);
System.out.println(b2.equals(b1));
System.out.println(b2.equals(b3));
System.out.println(b2.equals(b4));
System.out.println(b2.equals(b5));
System.out.println(b2.equals(b6));
System.out.println(b2.equals(b7));
System.out.println(b2.obtenerGrosor() == b1.obtenerGrosor() &
b2.obtenerColor() == b1.obtenerColor());
System.out.println(b2.obtenerGrosor() == b3.obtenerGrosor() &
b2.obtenerColor() == b3.obtenerColor());
System.out.println(b2.obtenerGrosor() == b1.obtenerGrosor() &
b2.obtenerColor().equals(b1.obtenerColor()));
System.out.println(b2.obtenerGrosor() == b4.obtenerGrosor() &
b2.obtenerColor().equals(b4.obtenerColor()));
System.out.println(b2.obtenerGrosor() == b6.obtenerGrosor() &
b2.obtenerColor().equals(b6.obtenerColor()));
System.out.println(b2.obtenerGrosor() == b7.obtenerGrosor() &
b2.obtenerColor().equals(b7.obtenerColor()));
```

- d. Analice si las siguientes afirmaciones son falsas o verdaderas luego de ejecutarse las asignaciones anteriores:
- b2 y b3 tienen la misma identidad
  - b2 y b3 son equivalentes
  - b2 y b3 están asociados a un mismo objeto de clase *Color*
  - b2 y b4 están asociados a un mismo objeto de clase *Color*
- e. Muestre la evolución del diagrama de objetos y la salida del segmento de código anterior, considerando que la clase *Borde* implementa:
- clonación en profundidad, igualdad superficial;
  - clonación superficial, igualdad en profundidad;
  - clonación en profundidad, igualdad en profundidad.
- f. Muestre la salida del siguiente segmento de código, asumiendo que se han ejecutado todas las instrucciones anteriores con los tres métodos implementados en forma superficial:

```
c1.establecerAzul(112);
System.out.println(c1 == c3+" "+ c1 == c4);
System.out.println(c1.equals(c3) +" "+ c1.equals(c4));
System.out.println(b2 == b6+" "+ b2 == b7);
System.out.println(b2.equals(b6));
System.out.println(b2.equals(b7));
```

```
System.out.println(b2.obtenerGrosor() == b6.obtenerGrosor() &
    b2.obtenerColor().equals(b6.obtenerColor()));
System.out.println(b2.obtenerGrosor() == b7.obtenerGrosor() &
    b2.obtenerColor().equals(b7.obtenerColor()));
```

- g. Asumiendo *copy* implementado en forma superficial y *clone* en profundidad, indique qué secuencia o secuencias de instrucciones corresponden al diagrama:



|   |                                                                                                                                        |
|---|----------------------------------------------------------------------------------------------------------------------------------------|
| A | <code>Borde b1 = new Borde (2, new Color (100,110,90));<br/>Borde b2 = b1;</code>                                                      |
| B | <code>Borde b1 = new Borde (2, new Color (100,110,90));<br/>Borde b2 = new Borde (2, new Color (120,110,120));<br/>b2.copy(b1);</code> |
| C | <code>Borde b1 = new Borde (2, new Color (100,110,90));<br/>Borde b2 = b1.clone();</code>                                              |
| D | <code>Borde b1 = new Borde (2, new Color (100,110,90));<br/>Borde b2 = new Borde (2, new Color (100,110,90));</code>                   |

**EJERCICIO 5.** Dadas las clases *A*, *B* y *C* definidas como sigue:

|                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> class A{     private B b1;     private int b2;     public A (B p1, int p2) {         b1 = p1;         b2 = p2;     }     public B obtenerB1(){         return b1;     }     public int obtenerB2(){         return b2;     }     public A clone () {         return new A(b1.clone(), b2);     }     public boolean equals(A p) {         return(b1.equals(p.obtenerB1())             &amp;&amp; b2 == p.obtenerB2());     } } </pre> | <pre> class B{     private C c1;     private char c2;     public B (C p1, char p2) {         c1 = p1;         c2 = p2;     }     public C obtenerC1() {         return c1;     }     public char obtenerC2() {         return c2;     }     public B clone () {         return new B(c1,c2);     }     public boolean equals(B p) {         return (c1 == p.obtenerC1()             &amp;&amp; c2 == p.obtenerC2());     } } </pre> | <pre> class C{     private int d1;     private char d2;     public C (int p1, char p2) {         d1 = p1;         d2 = p2;     }     public int obtenerD1() {         return d1;     }     public char obtenerD2() {         return d2;     }     public C clone () {         return new C (d1,d2);     }     public boolean equals(C p) {         return (d1 == p.obtenerD1()             &amp;&amp; d2 == p.obtenerD2());     } } </pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|





## Introducción a la Programación Orientada a Objetos DCIC – UNS – 2025



Y las instrucciones:

```
C vc = new C (10, 'a');  
B vb1 = new B (vc, '5');  
B vb2 = new B (vc.clone(), '5');  
A va1 = new A (vb1.clone(), 1);  
A va2 = new A (vb1.clone(), 1);  
A va3 = new A (vb2, 1);  
A va4 = new A (new B (new C(11, 'b'), 'c'), -1);
```

- Elabore el diagrama de objetos luego de ejecutarse la secuencia de instrucciones anterior.
- Agregue e implemente el método *copy* en las clases *A*, *B* y *C*, en profundidad en la clase *A* y superficial en *B*.
- Indique los valores de verdad de las siguientes expresiones:

`va1.equals(va2)`

`va1.equals(va3)`

**EJERCICIO 6.** Dado el siguiente diagrama:

| Jugador                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Equipo                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>&lt;&lt;Atributos de instancia&gt;&gt;<br/>nombre: String<br/>nroCamiseta: entero<br/>posicion: entero<br/>golesConvertidos: entero<br/>partidosJugados: entero</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | <p>&lt;&lt;Atributos de instancia&gt;&gt;<br/>nombre: String<br/>capitan: Jugador<br/>pG, pE, pP: entero<br/>gFavor, gContra: entero</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <p>&lt;&lt;Constructor&gt;&gt;<br/>Jugador(nom: String)<br/>&lt;&lt;Comandos&gt;&gt;<br/>establecerNroCamiseta(n: entero)<br/>establecerPosicion(n: entero)<br/>establecerGolesConvertidos(n: entero)<br/>establecerPartidosJugados(n: entero)<br/>aumentarGoles(n: entero)<br/>aumentarUnPartido()<br/>&lt;&lt;Consultas&gt;&gt;<br/>obtenerNombre(): String<br/>obtenerNroCamiseta(): entero<br/>obtenerPosicion(): entero<br/>obtenerGolesConvertidos(): entero<br/>obtenerPartidosJugados(): entero<br/>promedioGolesXPart(): entero<br/>masGoles(j: Jugador): boolean<br/>jugConMasGoles(j: Jugador): Jugador<br/>toString(): String<br/>clone(): Jugador<br/>equals(j: Jugador): boolean</p> | <p>&lt;&lt;Constructor&gt;&gt;<br/>Equipo(nom: String, cap: Jugador)<br/>&lt;&lt;Comandos&gt;&gt;<br/>incrementarPG(jugoElCap: boolean)<br/>incrementarPE(jugoElCap: boolean)<br/>incrementarPP(jugoElCap: boolean)<br/>aumentarGFavor(total, delCap: entero)<br/>aumentarGContra(total: entero)<br/>&lt;&lt;Consultas&gt;&gt;<br/>obtenerNombre():String<br/>obtenerCapitan():Jugador<br/>obtenerPG(): entero<br/>obtenerPE(): entero<br/>obtenerPP():entero<br/>obtenerGFavor():entero<br/>obtenerGContra():entero<br/>obtenerPartidos(): entero<br/>obtenerPuntos(): entero<br/>mejorPuntaje(e: Equipo): Equipo<br/>capitanConMasGoles(e: Equipo): Jugador<br/>toString(): String<br/>equals(e: Equipo): boolean</p> |

- *Equipo(nom: String, cap: Jugador)*. Requiere nom y cap ligados
- *incrementarPG(jugoElCap: boolean)*, *incrementarPE(jugoElCap: boolean)*, *incrementarPP(jugoElCap: boolean)*. Aumentan en 1 los partidos del equipo y si corresponde envía un mensaje al capitán para que incremente en 1 sus partidos.
- *aumentarGfavor(total, delCap: entero)*. Aumenta los goles del equipo y si corresponde envía un mensaje al capitán para actualizar sus goles. *delCap* indica cuántos goles marcados son del capitán.
- *obtenerPuntos(): entero*. Se computa considerando que por cada partido ganado se obtienen 3 puntos y por cada empate se logra 1.
- *obtenerPartidos(): entero*. La cantidad de partidos es la suma de los partidos ganados, perdidos y empatados.
- *mejorPuntaje(e: Equipo): Equipo*. Retorna el equipo con más puntaje, si los dos equipos tienen los mismos puntos, devuelve el que tiene mayor cantidad de goles a favor y si también hay coincidencia, el que tiene menos goles en contra. Si hay coincidencia devuelve el equipo que recibe el mensaje.
- *equals(e: Equipo): boolean*. Requiere e ligado, se implementa en profundidad.

- Implemente la clase *Equipo*
- Considerando el mismo modelo para *Jugador* implemente una nueva clase *Equipo* para la siguiente especificación:
  - *incrementarPG(jugoElCap: boolean)*, *incrementarPE(jugoElCap: boolean)*, *incrementarPP(jugoElCap: boolean)*. Aumenta en 1 los partidos del equipo. Si jugó el capitán requiere que la clase cliente incremente el número de partidos jugados por el capitán.
  - *incrementarGFavor(total: entero)*. Aumenta los goles del equipo. Si el gol fue del capitán requiere que la clase cliente aumente el número de goles del capitán.
  - *equals(e: Equipo): boolean*. Si e no está ligado retorna false, se implementa superficial.
- Verifique cada implementación de acuerdo al *Tester* publicado.

#### **EJERCICIO 7.** Analice si las siguientes afirmaciones son correctas:

- Una clase A está asociada a la clase B si tiene uno o más atributos de la clase B.
- Si una clase A está asociada a una clase B, el ambiente de referenciamiento de un método definido en la clase A incluye a todos los atributos y servicios definidos en B.
- Si una clase A está asociada a una clase B y A incluye un método p, la clase B no puede definir un método p con el mismo número y tipo de parámetros que el método p definido en A.
- Existe una relación de dependencia entre las clases A y B, si A declara una variable local, un parámetro o retorna un resultado de clase B.



Introducción a la Programación Orientada a Objetos  
DCIC – UNS – 2025



**EJERCICIO 8.** Implemente y verifique con una clase *Tester* el siguiente diagrama de clases:

| Vehículo                                                                                                                                                                                                                                                                                                                                                    | Hora                                                                                                                                                                                                                                                                                                            | Tarifa                                                                                                                                                                |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <<Atributos de instancia>><br>ingreso: Hora<br>egreso: Hora<br>numero: entero<br>patente: String                                                                                                                                                                                                                                                            | <<Atributos de instancia>><br>hor: entero<br>min: entero                                                                                                                                                                                                                                                        | <<Atributos de instancia>><br>t15, t30, t60, tFija: entero                                                                                                            |
| <Constructor><br>Vehículo (i: Hora, n: entero, p: String)<br><<Comandos>><br>egresaVehiculo(c: Hora)<br>copy(v: Vehiculo)<br><<Consultas>><br>obtenerIngreso(): Hora<br>obtenerEgreso(): Hora<br>obtenerNumero(): entero<br>obtenerPatente(): String<br>obteneraCobrar(t: Tarifa): entero<br>equals(c: Vehículo): boolean<br>anterior(v: Vehículo): boolean | <Constructor><br>Hora(h: entero, m: entero)<br><<Comandos>><br>establecerHora(c: entero)<br>establecerMinutos(c: entero)<br>copy(h: Hora)<br><<Consultas>><br>obtenerHora(): entero<br>obtenerMinutos(): entero<br>equals(c: Hora): boolean<br>diferenciaMinutos(c: Hora): entero<br>anterior(c: Hora): boolean | <Constructor><br>Tarifa(t1, t2, t3, tfija: entero)<br><<Consultas>><br>obtenerT15(): entero<br>obtenerT30(): entero<br>obtenerT60(): entero<br>obtenerTFija(): entero |

En la clase *Hora*:

- *hor*, es un entero entre 7 y 22; *min*, es un entero entre 0 y 59. La clase cliente es responsable de garantizar que los parámetros del constructor estén dentro del rango especificado.
- *diferenciaMinutos(c: Hora): entero*. Retorna la diferencia calculada en minutos entre la hora y minutos del objeto que recibe el mensaje y la hora y minutos del parámetro *c*.
- *anterior(c: Hora): boolean*. Retorna true si el objeto que recibe el mensaje representa una hora anterior al parámetro *c*.
- *equals(c: Hora): boolean*. Requiere *c* ligado

En la clase *Vehículo*:

- El constructor establece el horario de ingreso distinto de null, la patente y el número de la cochera. Asigna null al horario de egreso.
- *egresaVehiculo(c: Hora)*. Establece la hora de egreso
- *obteneraCobrar(t: Tarifa): entero*. Si Hora de egreso es null retorna 0, sino calcula y retorna el monto a cobrar aplicando la tarifa que corresponda según la diferencia entre la hora de ingreso y la de egreso.
- *anterior(v: Vehículo): boolean*. Retorna true si el objeto que recibe el mensaje representa un Vehículo con horario de ingreso anterior al pasado por parámetro. Requiere *v* ligado.
- *equals(v: Vehículo): boolean* y *copy(v: Vehículo)*. Ambos requieren *v* ligado, y se implementan en profundidad, a excepción de la patente que se compara y copia en forma superficial. *Copy* requiere además que el vehículo ya haya egresado.

El monto a cobrar se calcula según los valores que se reciben en el parámetro de clase *Tarifa*, si el vehículo estuvo 15 minutos o menos se aplica la tarifa *t15*, 30 minutos o menos se aplica *t30*, 60 minutos o menos se aplica *t60* y más de 60 minutos se aplica la tarifa fija de día completo.

**EJERCICIO 9.** Implemente y verifique con la misma clase *Tester* que en el ejercicio anterior, considerando que el diagrama de la clase *Vehículo* es ahora:

| Vehículo                                                                                                                                                                                                                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <<Atributos de instancia>><br>ingreso: Hora<br>aCobrar: entero<br>numero: entero<br>patente: String                                                                                                                                                                                                                                  |
| <<Constructor>><br>Vehículo(i: Hora, n: entero, p: String)<br><<Comandos>><br>egresaVehiculo(c: Hora,t: Tarifa)<br>copy(v: Vehiculo)<br><<Consultas>><br>obtenerIngreso(): Hora<br>obtenerNumero(): entero<br>obtenerPatente(): String<br>obteneraCobrar(): entero<br>equals(c: Vehículo): boolean<br>anterior(v: Vehículo): boolean |

- El constructor establece el horario de ingreso distinto de null, la patente y el número de la cochera. Asigna 0 a aCobrar
- *egresaVehiculo(c: Hora,t: Tarifa)*. Calcula y modifica el atributo aCobrar a partir de la hora de ingreso, el parámetro c que establece la hora de egreso y la tarifa que corresponde según el parámetro t.

**Observen que no solo ha cambiado la funcionalidad de algunos métodos sino también la signatura.**

#### EJERCICIO 10.

- Explicá la diferencia entre *copy* superficial y en profundidad, proponé un ejemplo simple y un diagrama de objetos para ilustrar esta diferencia.
- De manera similar, explicá la diferencia entre *equals* superficial y en profundidad, utilizando un ejemplo simple y un diagrama de objetos para clarificar la distinción.
- Describa las relaciones de dependencia y asociación entre clases. Muestre las dos relaciones en un único ejemplo ilustrativo de su utilidad.